FILE-SYSTEM INTERFACE

DR NISHANT JAIN
MITS-DU GWALIOR

FILE-SYSTEM INTERFACE

- File Concept
- Access Methods
- Directory Structure
- File-System Mounting
- File Sharing
- Protection

OBJECTIVES

- To explain the function of file systems
- To describe the interfaces to file systems
- To discuss file-system design tradeoffs, including access methods, file sharing, file locking, and directory structures
- To explore file-system protection

FILE CONCEPT

- Contiguous logical address space
- Types:
  - Data
    - numeric
    - character
    - binary
  - Program

FILE STRUCTURE

- None - sequence of words, bytes
- Simple record structure
  - Lines
  - Fixed length
  - Variable length
- Complex Structures
  - Formatted document
  - Relocatable load file
- Can simulate last two with first method by inserting appropriate control characters
- Who decides:
  - Operating system
  - Program

FILE ATTRIBUTES

- **Name** – only information kept in human-readable form
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk

FILE OPERATIONS

- File is an **abstract data type**
- **Create**
- **Write**
- **Read**
- **Reposition within file**
- **Delete**
- **Truncate**
- *Open(F_i)* – search the directory structure on disk for entry F_i, and move the content of entry to memory
- *Close (F_i)* – move the content of entry F_i in memory to directory structure on disk

OPEN FILES

- Several pieces of data are needed to manage open files:
  - File pointer: pointer to last read/write location, per process that has the file open
  - File-open count: counter of number of times a file is open – to allow removal of data from open-file table when last processes closes it
  - Disk location of the file: cache of data access information
  - Access rights: per-process access mode information

OPEN FILE LOCKING

- Provided by some operating systems and file systems
- Mediates access to a file
- Mandatory or advisory:
  - **Mandatory** – access is denied depending on locks held and requested
  - **Advisory** – processes can find status of locks and decide what to do

FILE TYPES – NAME, EXTENSION

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com-pressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

ACCESS METHODS

- **Sequential Access**

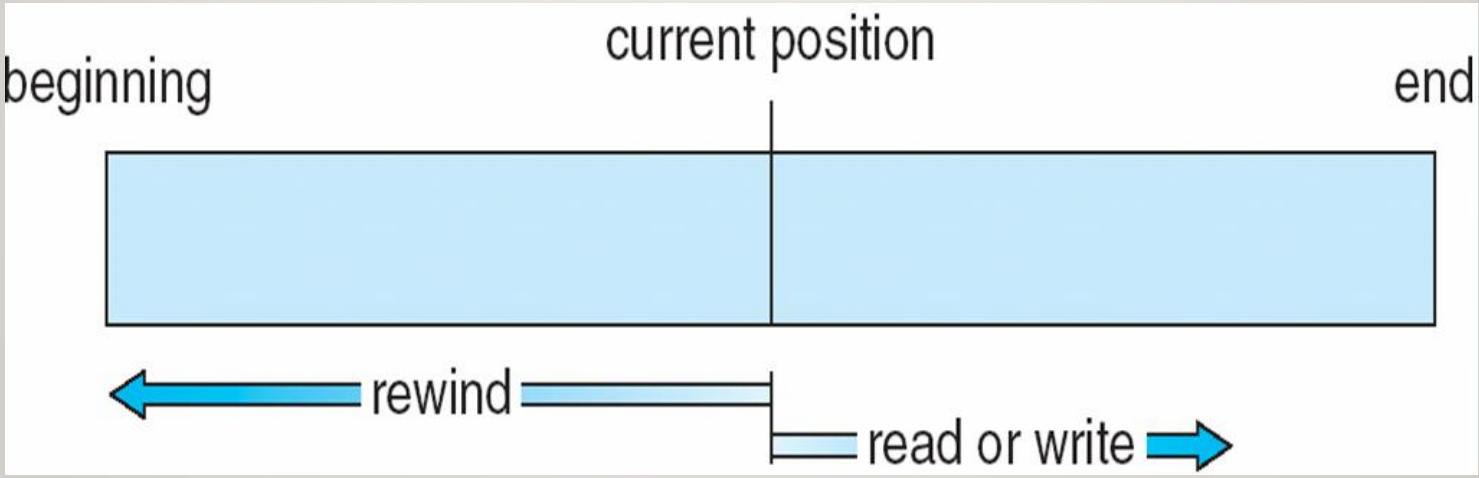
  read next
  write next
  reset
  no read after last write
  (rewrite)

- **Direct Access**

  read *n*
  write *n*
  position to *n*
  read next
  write next
  rewrite *n*

n = relative block number

SEQUENTIAL-ACCESS FILE



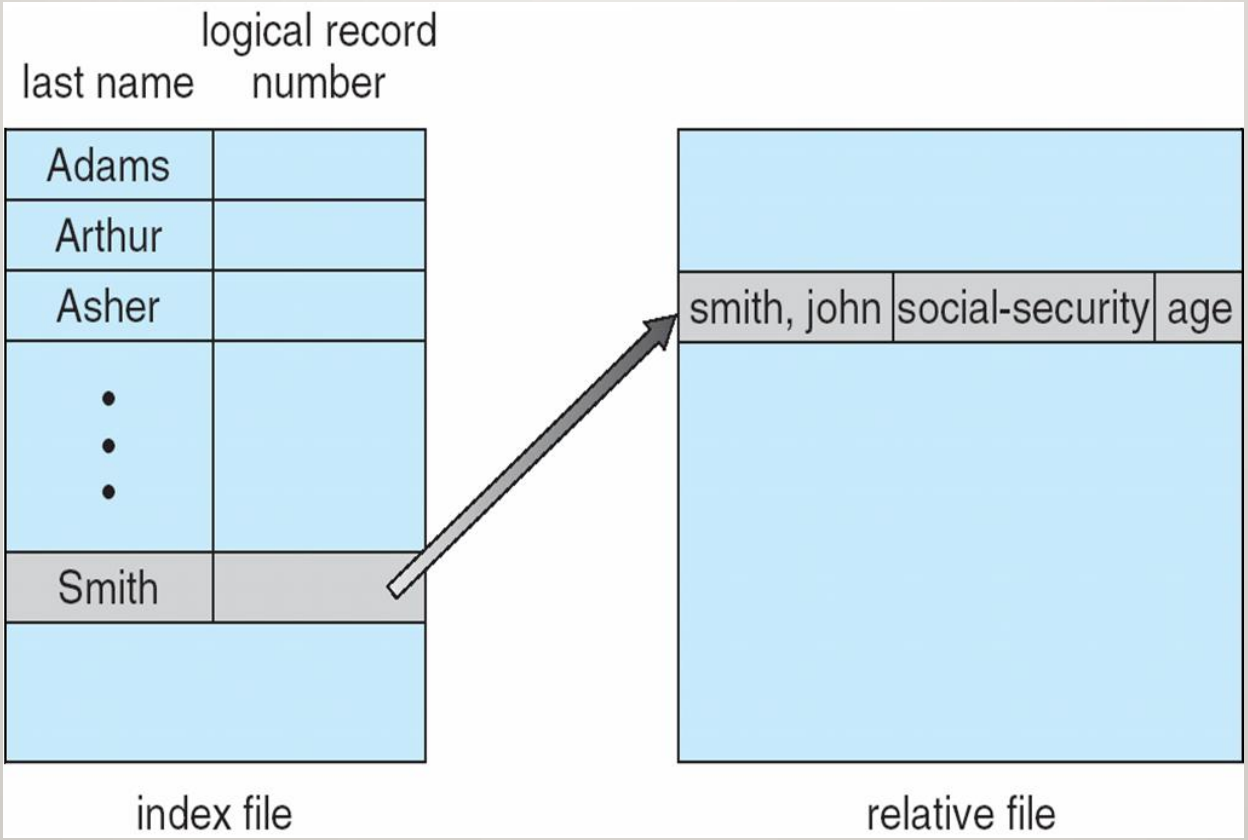
SIMULATION OF SEQUENTIAL ACCESS ON DIRECT-ACCESS FILE

Dr Nishant Jain MITS DU Gwalior

sequential access	implementation for direct access
reset	*cp* = 0;
read next	*read cp*; *cp* = *cp* + 1;
write next	*write cp*; *cp* = *cp* + 1;

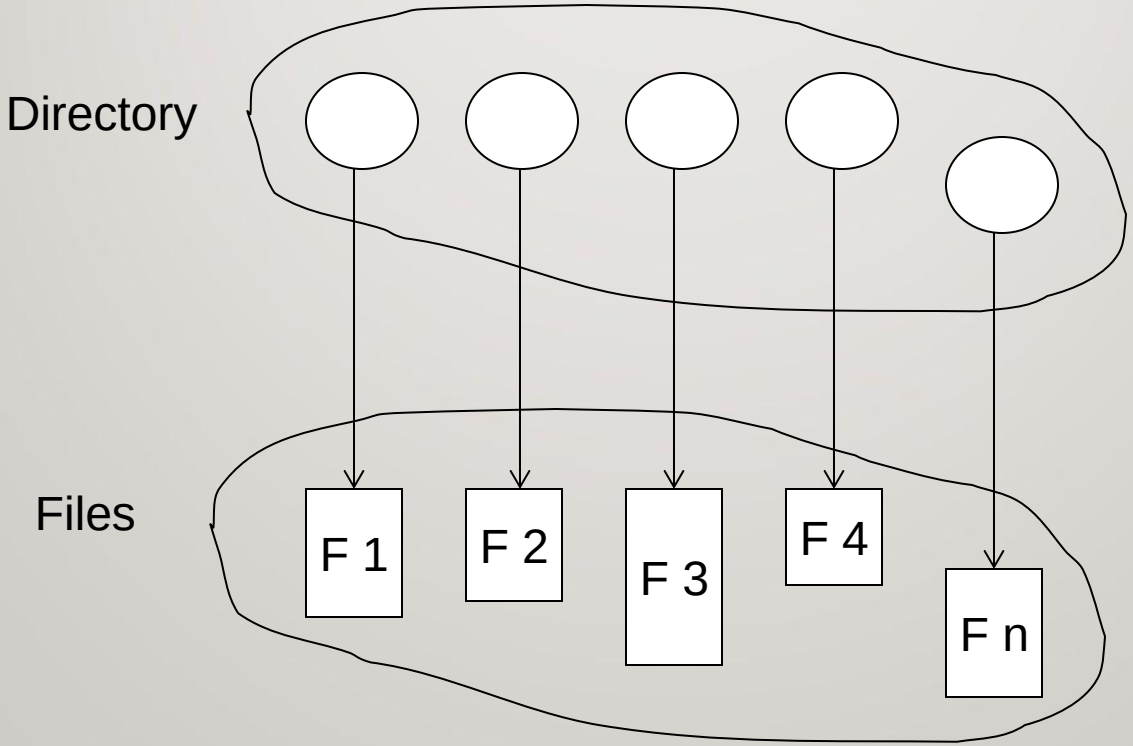
EXAMPLE OF INDEX AND RELATIVE FILES

Dr. Nishant Jain MITS DU Gwalior



DIRECTORY STRUCTURE

- A collection of nodes containing information about all files



Both the directory structure and the files reside on disk
Backups of these two structures are kept on tapes

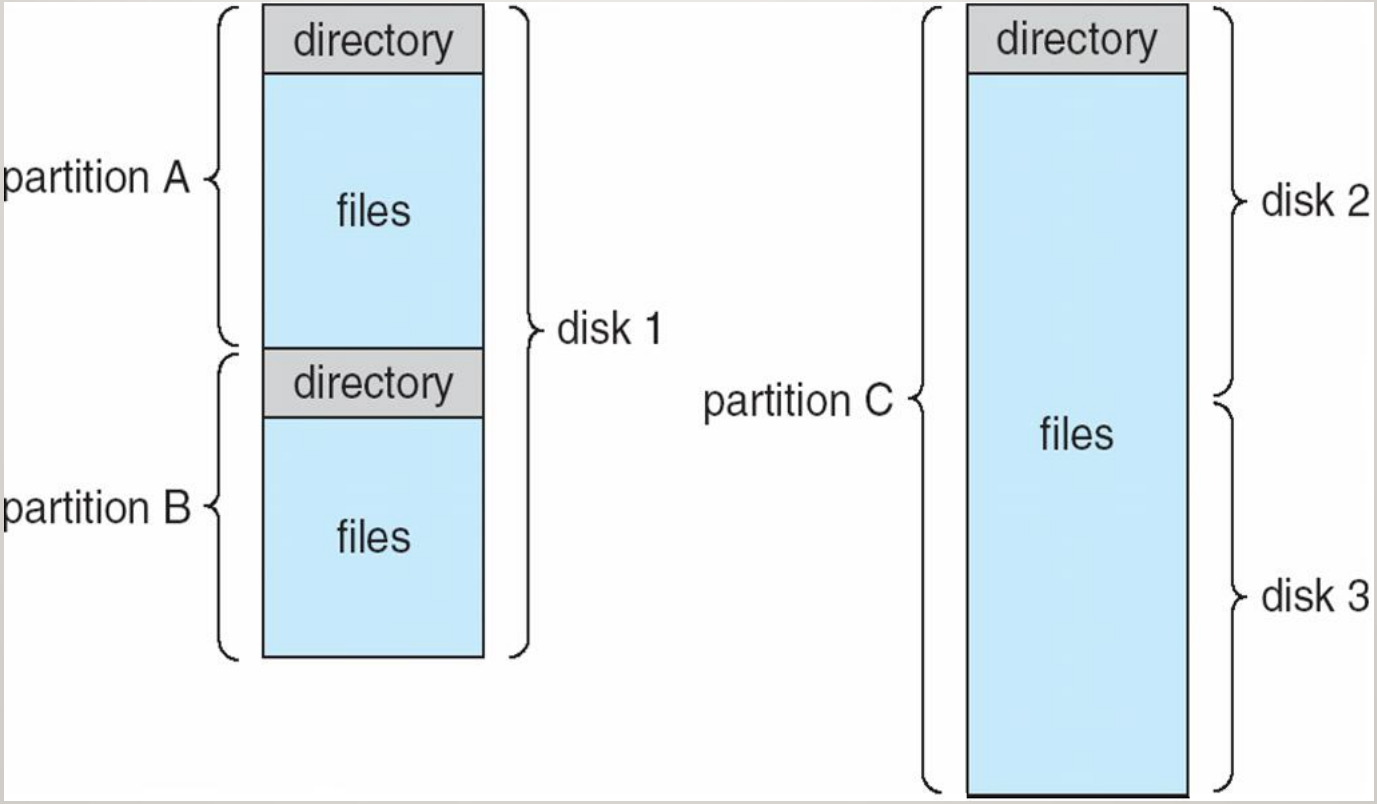
DISK STRUCTURE

Dr Nishant Jain MITS DU Gwalior

- Disk can be subdivided into partitions
- Disks or partitions can be RAID protected against failure
- Disk or partition can be used raw – without a file system, or formatted with a file system
- Partitions also known as minidisks, slices
- Entity containing file system known as a volume
- Each volume containing file system also tracks that file system's info in device directory or volume table of contents
- As well as general-purpose file systems there are many special-purpose file systems, frequently all within the same operating system or computer

A TYPICAL FILE-SYSTEM ORGANIZATION

Dr Nishant Jain MITS DU Gwalior



OPERATIONS PERFORMED ON DIRECTORY

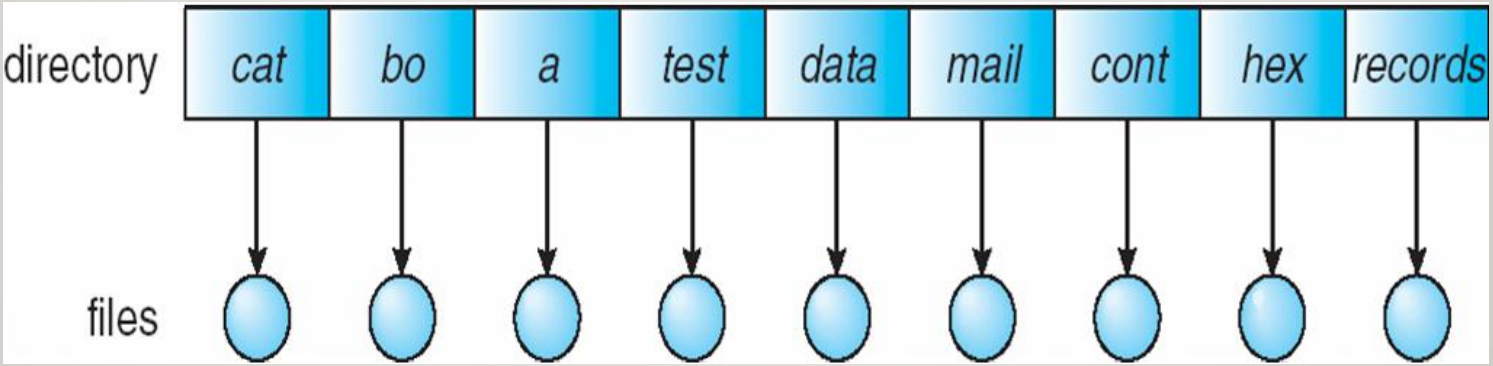
- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

ORGANIZE THE DIRECTORY (LOGICALLY) TO OBTAIN

- Efficiency – locating a file quickly
- Naming – convenient to users
  - Two users can have same name for different files
  - The same file can have several different names
- Grouping – logical grouping of files by properties, (e.g., all Java programs, all games, …)

SINGLE-LEVEL DIRECTORY

- A single directory for all users

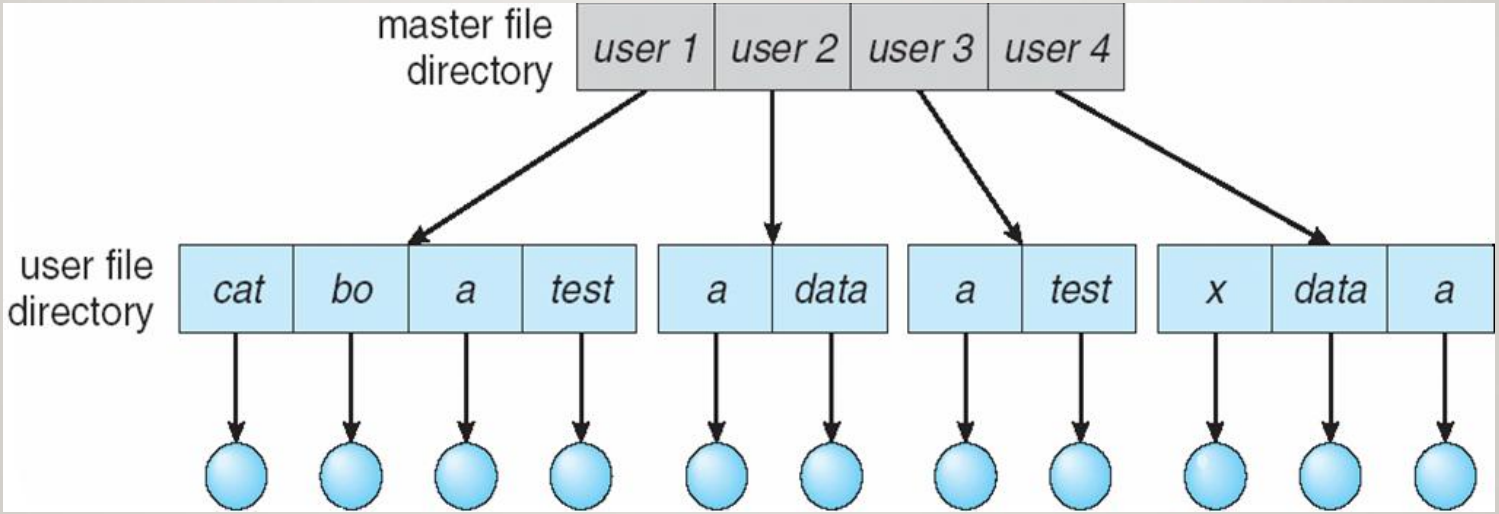


Naming problem

Grouping problem

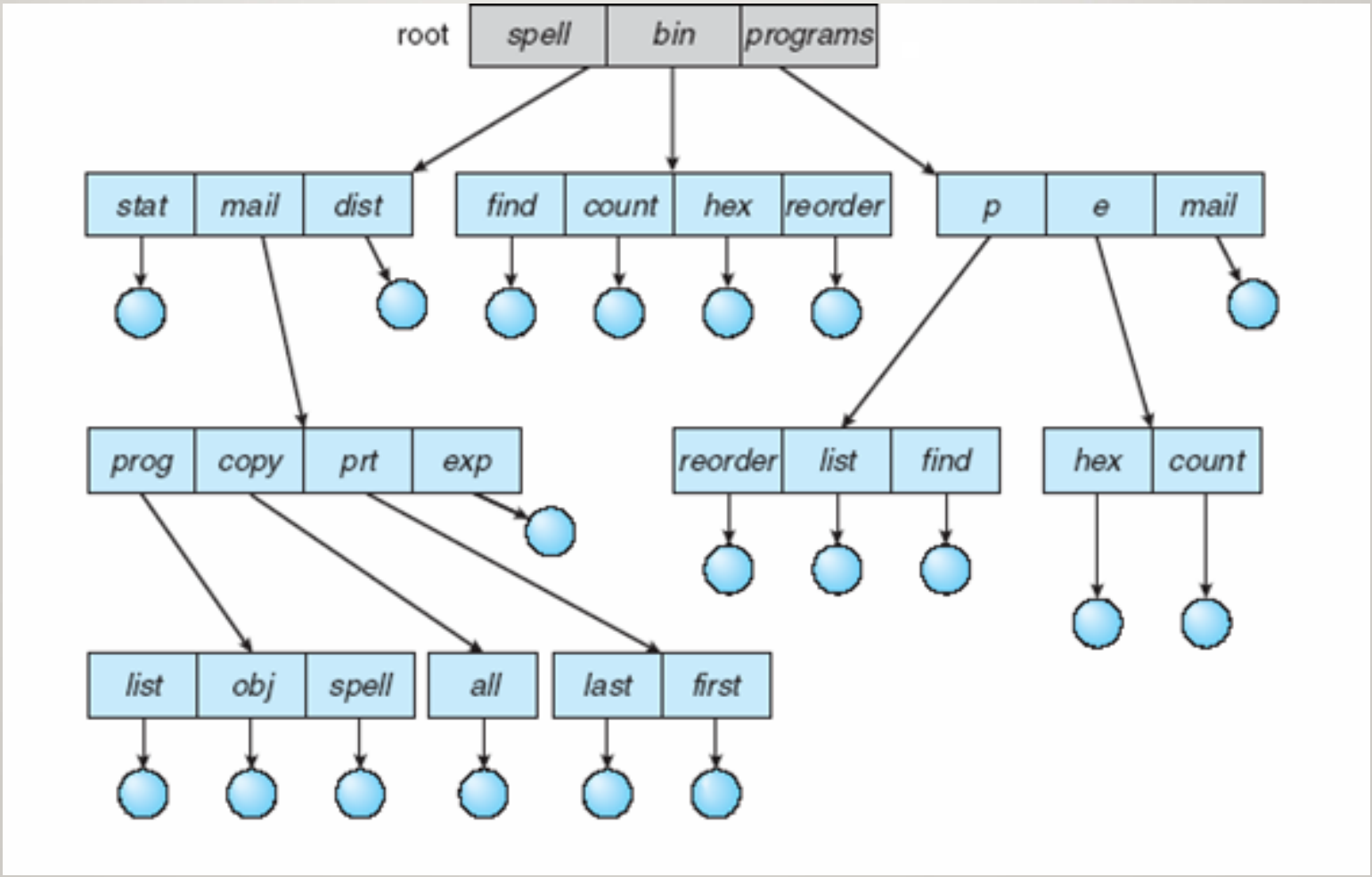
TWO-LEVEL DIRECTORY

- Separate directory for each user



- Path name
- Can have the same file name for different user
- Efficient searching
- No grouping capability

TREE-STRUCTURED DIRECTORIES



TREE-STRUCTURED DIRECTORIES (CONT)

- Efficient searching
- Grouping Capability
- Current directory (working directory)
  - cd /spell/mail/prog
  - type list

TREE-STRUCTURED DIRECTORIES (CONT)

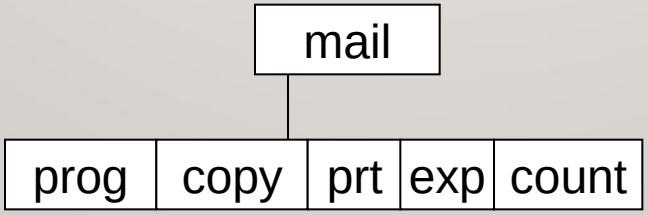
- **Absolute** or **relative** path name
- Creating a new file is done in current directory
- Delete a file

  rm <file-name>
- Creating a new subdirectory is done in current directory

  mkdir <dir-name>

  Example: if in current directory /mail

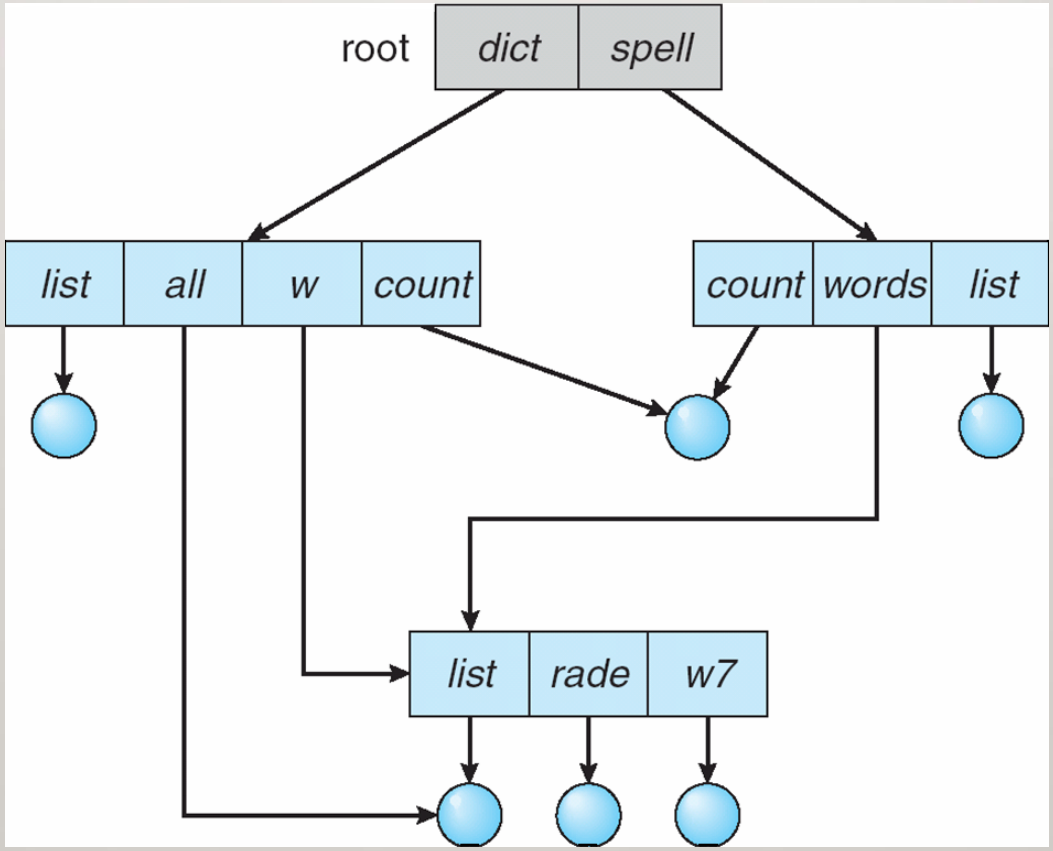
  mkdir count



Deleting “mail” ⇒ deleting the entire subtree rooted by “mail”

ACYCLIC-GRAPH DIRECTORIES

- Have shared subdirectories and files



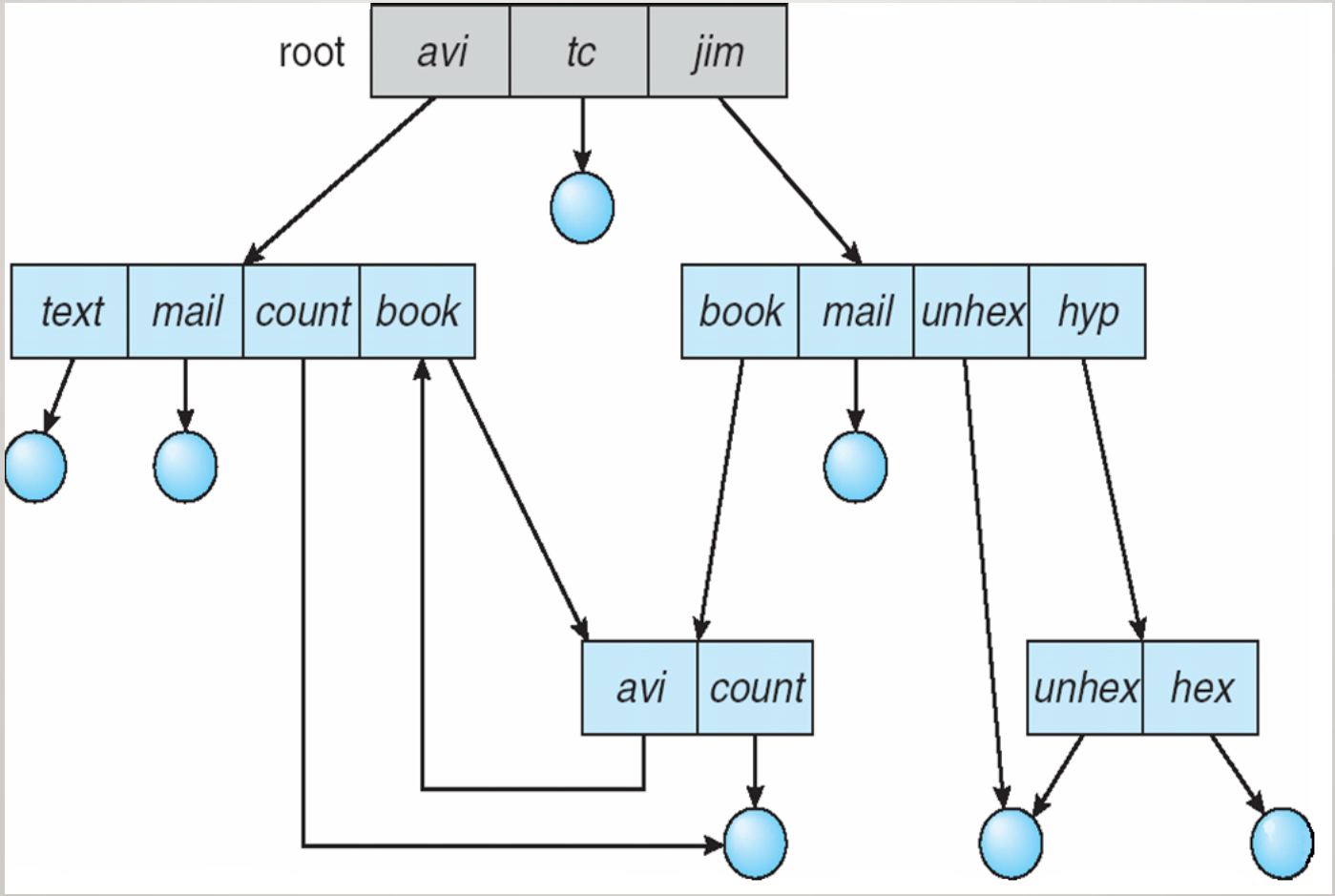
ACYCLIC-GRAPH DIRECTORIES (CONT.)

- Two different names (aliasing)
- If *dict* deletes *list* ⇒ dangling pointer

  Solutions:

  - Backpointers, so we can delete all pointers
    Variable size records a problem
  - Backpointers using a daisy chain organization
  - Entry-hold-count solution
- New directory entry type
  - **Link** – another name (pointer) to an existing file
  - **Resolve the link** – follow pointer to locate the file

GENERAL GRAPH DIRECTORY



GENERAL GRAPH DIRECTORY (CONT.)

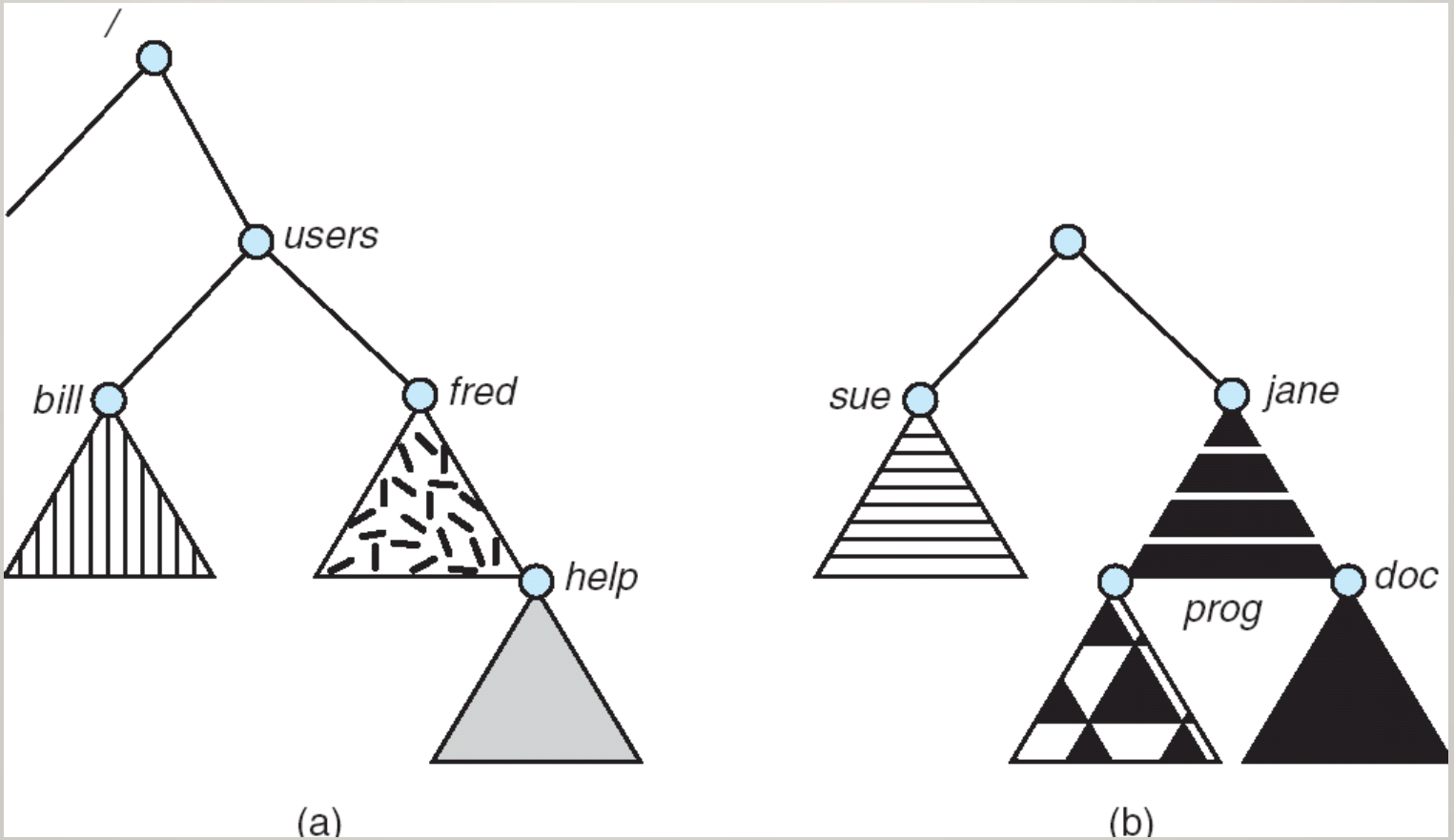
- How do we guarantee no cycles?
  - Allow only links to file not subdirectories
  - Garbage collection
  - Every time a new link is added use a cycle detection
    algorithm to determine whether it is OK

FILE SYSTEM MOUNTING

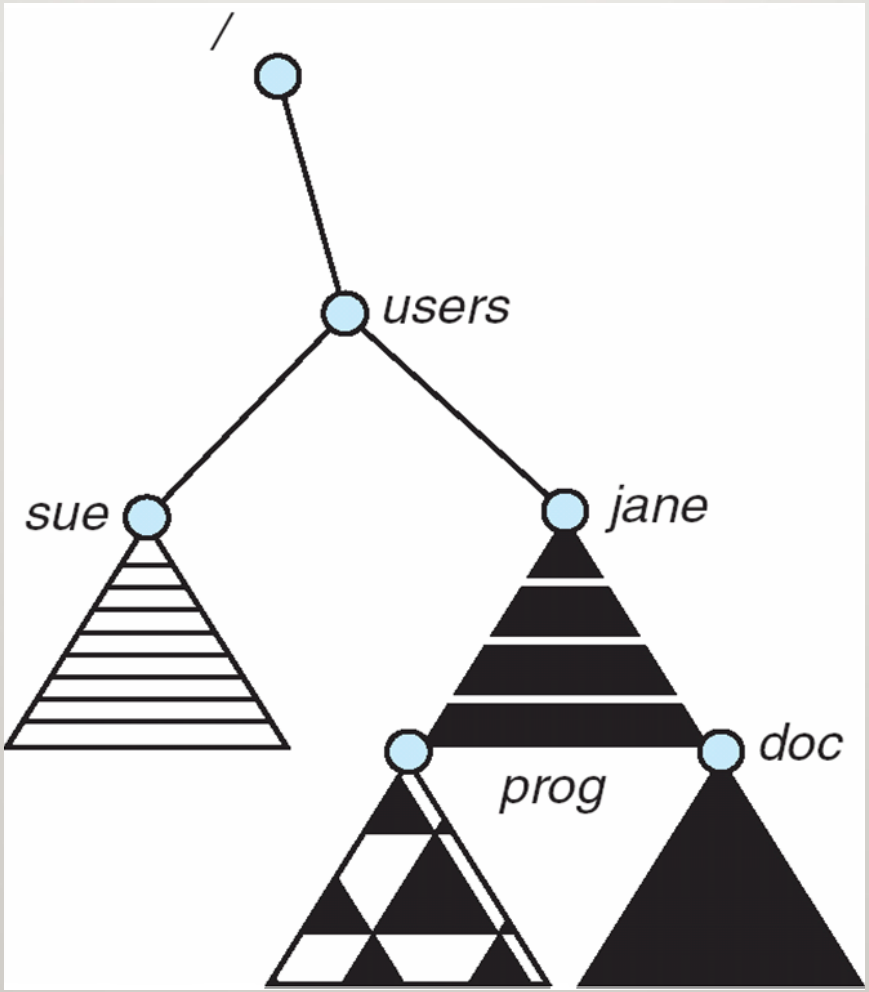
- A file system must be **mounted** before it can be accessed
- A unmounted file system (i.e. Fig. 11-11(b)) is mounted at a **mount point**

(A) EXISTING. (B) UNMOUNTED PARTITION

Dr Nishant Jain MITS DU Gwalior



MOUNT POINT



FILE SHARING

- Sharing of files on multi-user systems is desirable
- Sharing may be done through a **protection** scheme
- On distributed systems, files may be shared across a network
- Network File System (NFS) is a common distributed file-sharing method

FILE SHARING – MULTIPLE USERS

- **User IDs** identify users, allowing permissions and protections to be per-user

- **Group IDs** allow users to be in groups, permitting group access rights

FILE SHARING – REMOTE FILE SYSTEMS

- Uses networking to allow file system access between systems
  - Manually via programs like FTP
  - Automatically, seamlessly using **distributed file systems**
  - Semi automatically via the **world wide web**
- **Client-server** model allows clients to mount remote file systems from servers
  - Server can serve multiple clients
  - Client and user-on-client identification is insecure or complicated
  - **NFS** is standard UNIX client-server file sharing protocol
  - **CIFS** is standard Windows protocol
  - Standard operating system file calls are translated into remote calls
- Distributed Information Systems **(distributed naming services)** such as LDAP, DNS, NIS, Active Directory implement unified access to information needed for remote computing

FILE SHARING – FAILURE MODES

- Remote file systems add new failure modes, due to network failure, server failure
- Recovery from failure can involve state information about status of each remote request
- Stateless protocols such as NFS include all information in each request, allowing easy recovery but less security

FILE SHARING – CONSISTENCY SEMANTICS

- **Consistency semantics** specify how multiple users are to access a shared file simultaneously
  - Similar to Ch 7 process synchronization algorithms
    - Tend to be less complex due to disk I/O and network latency (for remote file systems
  - Andrew File System (AFS) implemented complex remote file sharing semantics
  - Unix file system (UFS) implements:
    - Writes to an open file visible immediately to other users of the same open file
    - Sharing file pointer to allow multiple users to read and write concurrently
  - AFS has session semantics
    - Writes only visible to sessions starting after the file is closed

PROTECTION

- File owner/creator should be able to control:
  - what can be done
  - by whom

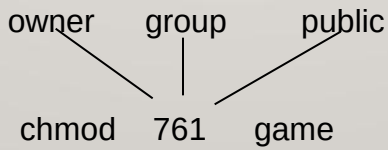
- Types of access
  - **Read**
  - **Write**
  - **Execute**
  - **Append**
  - **Delete**
  - **List**

ACCESS LISTS AND GROUPS

- Mode of access: read, write, execute
- Three classes of users

			RWX
a) **owner access**	7	⇒	1 1 1
b) **group access**	6	⇒	RWX 1 1 0
c) **public access**	1	⇒	RWX 0 0 1

- Ask manager to create a group (unique name), say G, and add some users to the group.
- For a particular file (say *game*) or subdirectory, define an appropriate access.

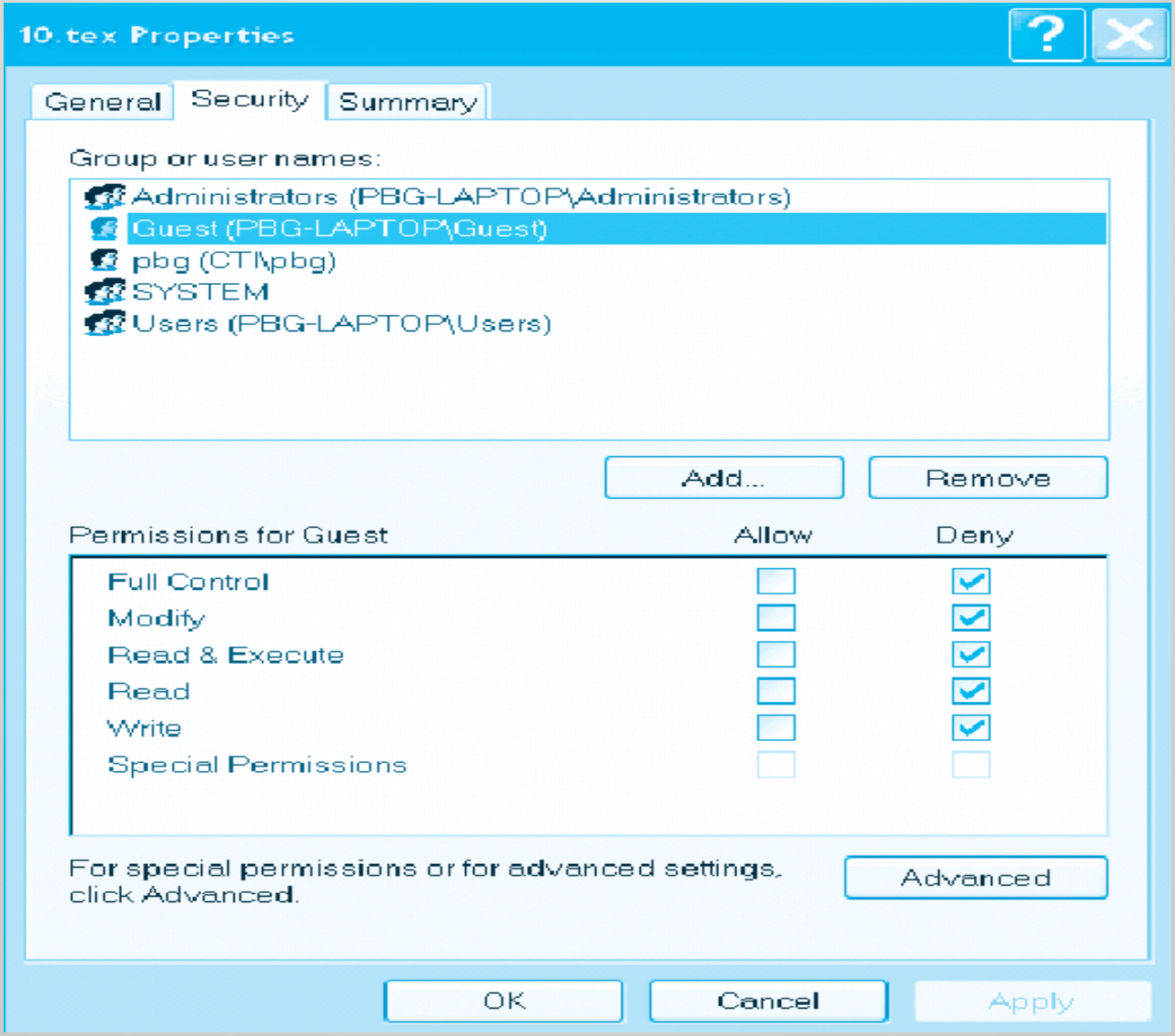


Attach a group to a file

chgrp G game

WINDOWS XP ACCESS-CONTROL LIST MANAGEMENT

Dr. Nishant Jain MITS DU Gwalior



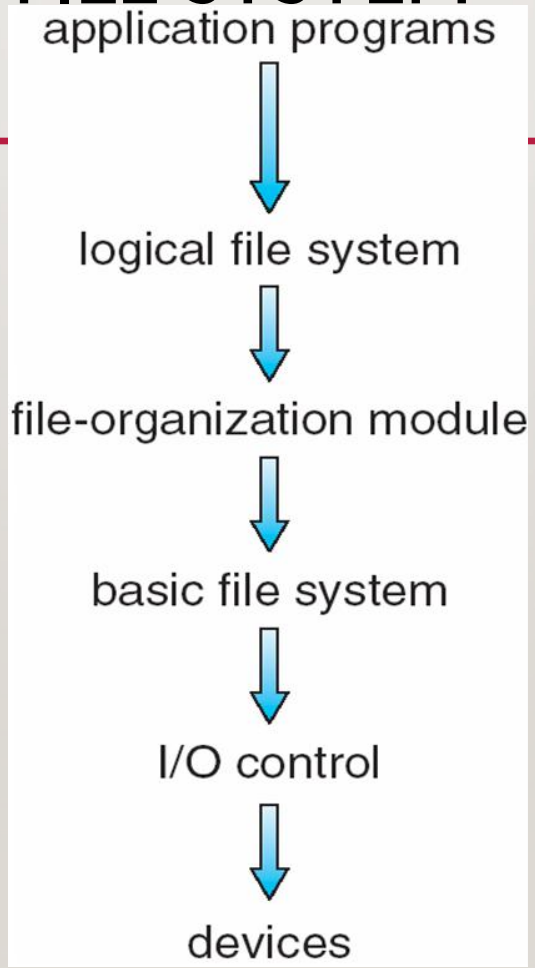
A SAMPLE UNIX DIRECTORY LISTING

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx------	5 pbg	staff	512	Jul 8 09.33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx------	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/

FILE-SYSTEM STRUCTURE

- File structure
  - Logical storage unit
  - Collection of related information
- File system organized into layers
- File system resides on secondary storage (disks)
  - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily
- File control block – storage structure consisting of information about a file
- Device driver controls the physical device

LAYERED FILE SYSTEM



FILE-SYSTEM IMPLEMENTATION

- Boot control block contains info needed by system to boot OS from that volume
- Volume control block contains volume details
- Directory structure organizes the files
- Per-file File Control Block (FCB) contains many details about the file

A TYPICAL FILE CONTROL BLOCK

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

ALLOCATION METHODS

- An allocation method refers to how disk blocks are allocated for files:

- Contiguous allocation

- Linked allocation

- Indexed allocation

CONTIGUOUS ALLOCATION

- Each file occupies a set of contiguous blocks on the disk

- Simple – only starting location (block #) and length (number of blocks) are required

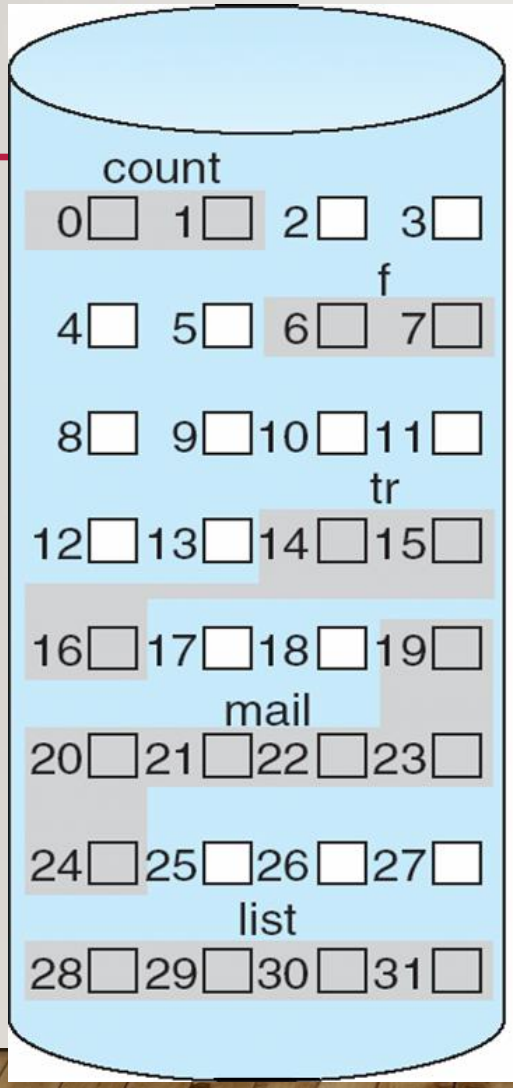
- Random access

- Wasteful of space (dynamic storage-allocation problem)

- Files cannot grow

CONTIGUOUS ALLOCATION OF DISK SPACE

Dr Nishant Jain MITS DU Gwalior



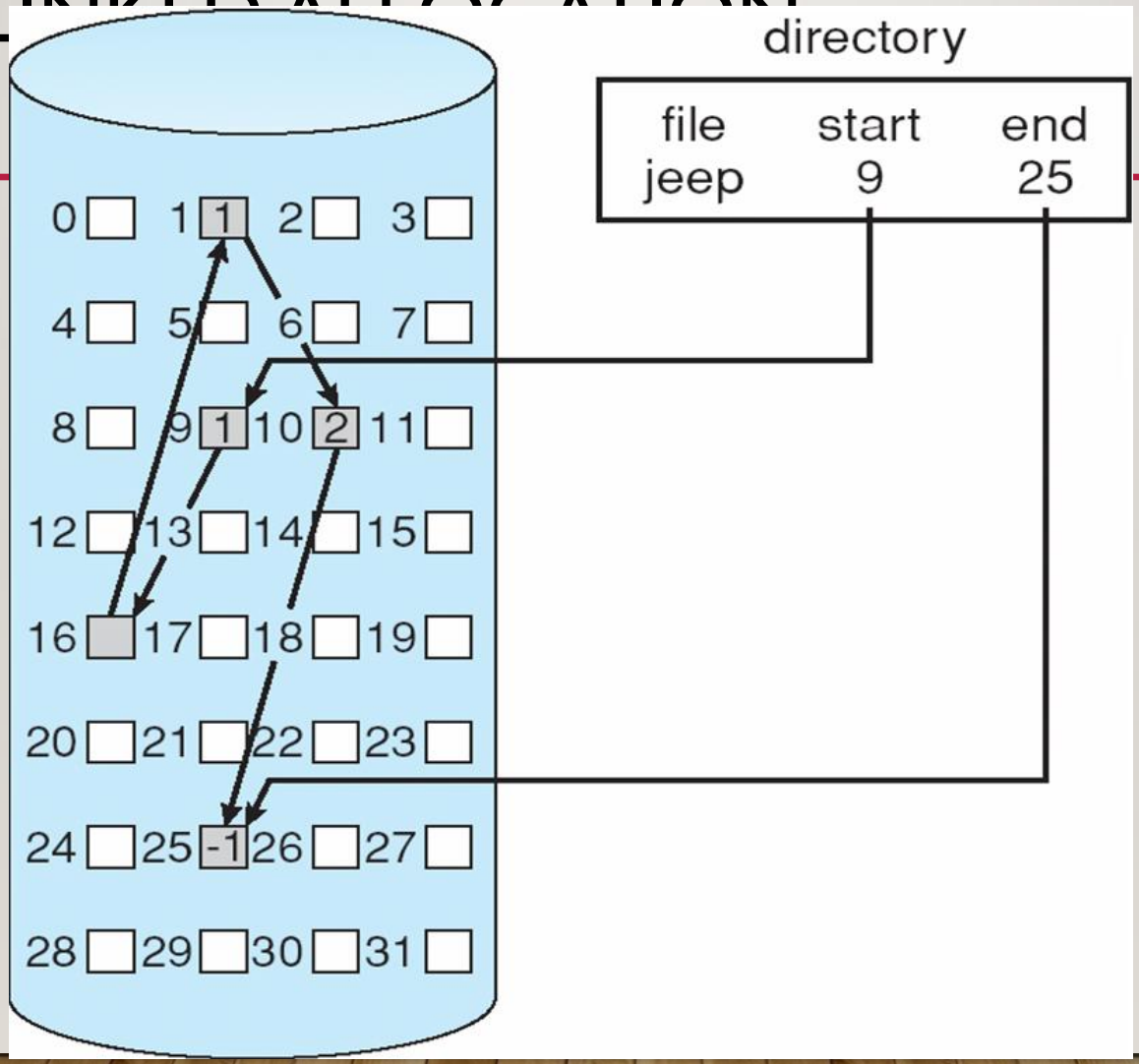
directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

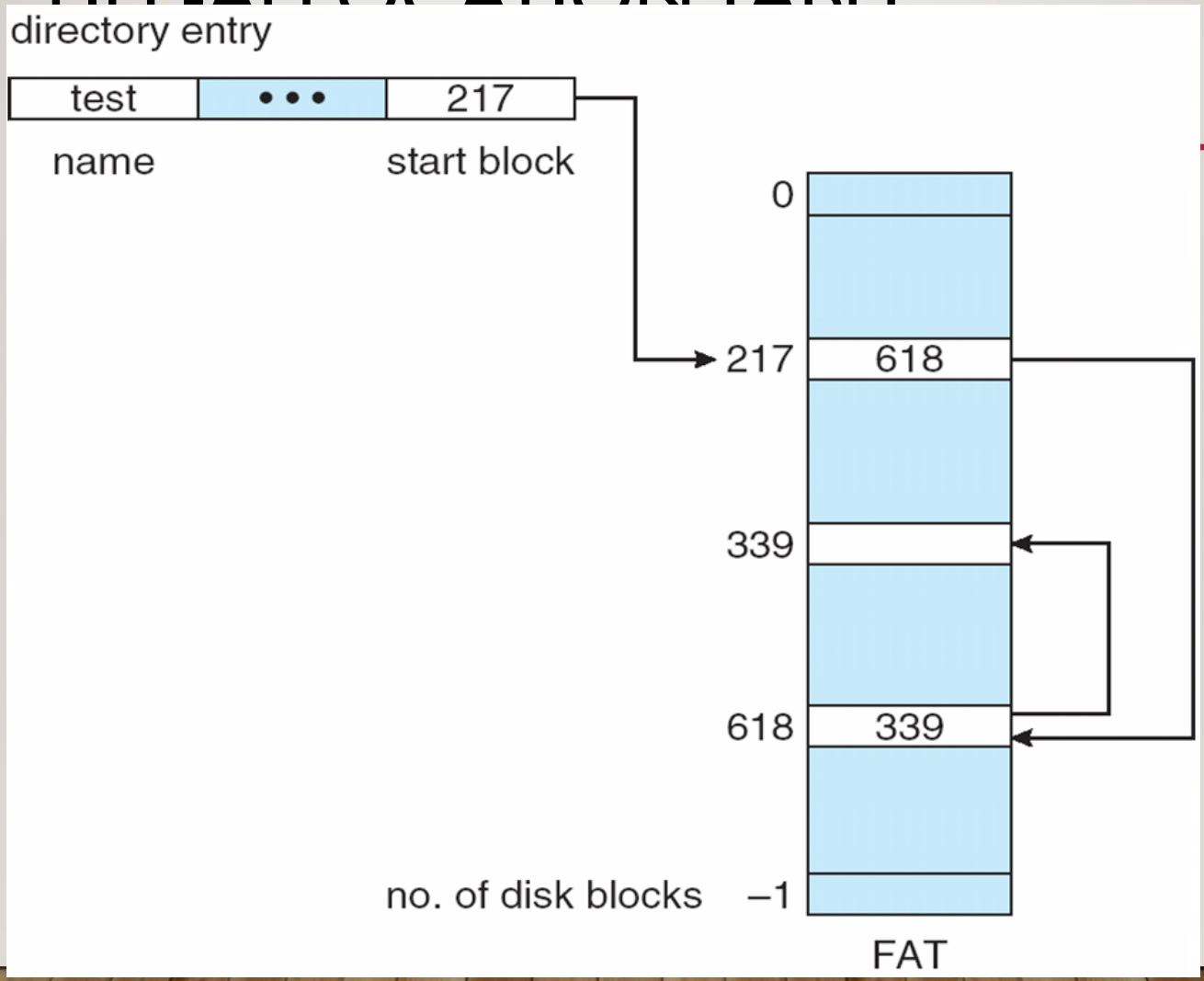
EXTENT-BASED SYSTEMS

- Many newer file systems (I.e. Veritas File System) use a modified contiguous allocation scheme
- Extent-based file systems allocate disk blocks in extents
- An extent is a contiguous block of disks
  - Extents are allocated for file allocation
  - A file consists of one or more extents

LINKED ALLOCATION

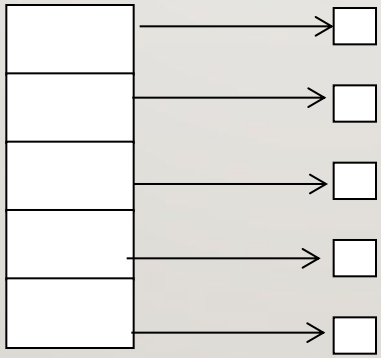


FILE-ALLOCATION TABLE



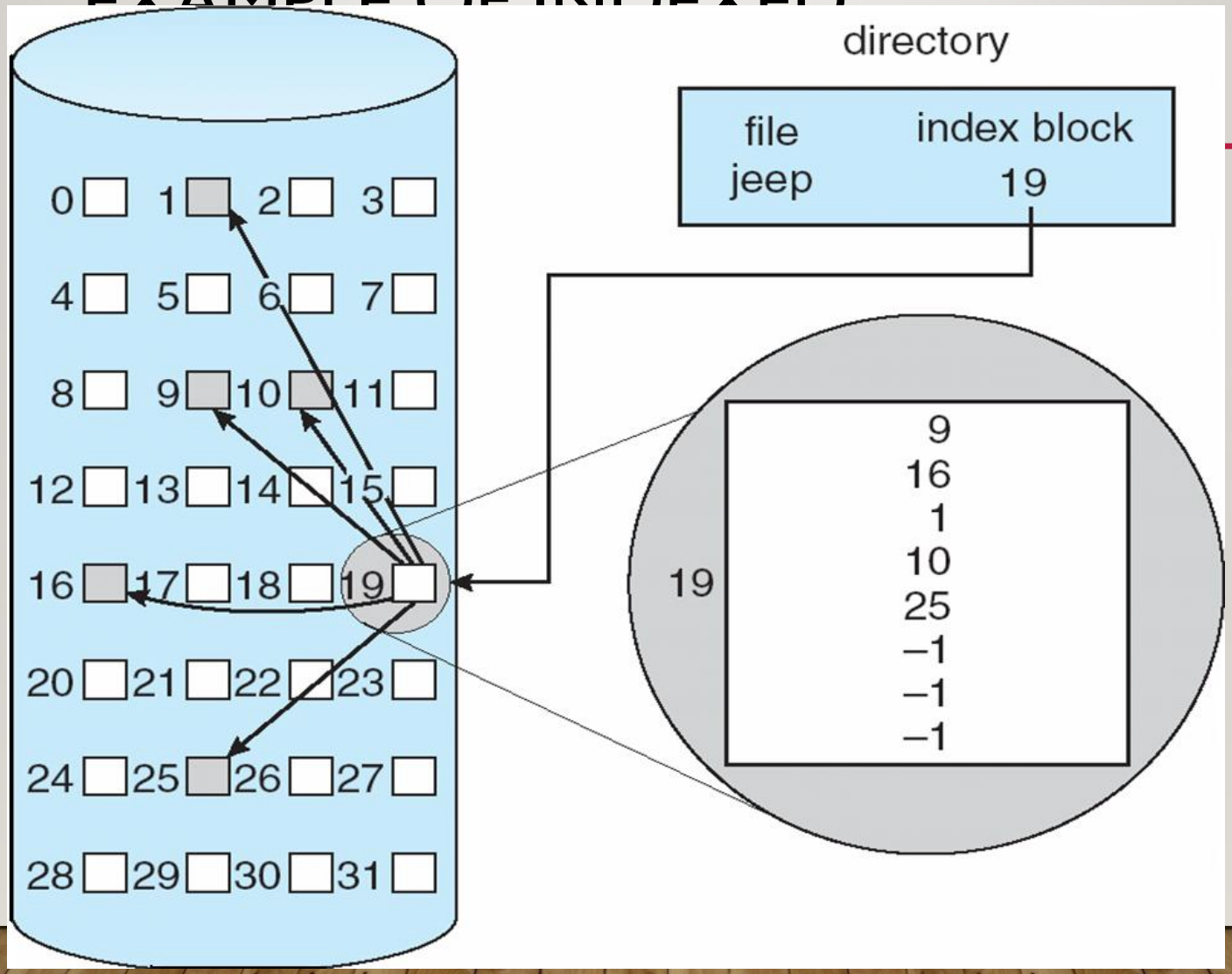
INDEXED ALLOCATION

- Brings all pointers together into the index block
- Logical view

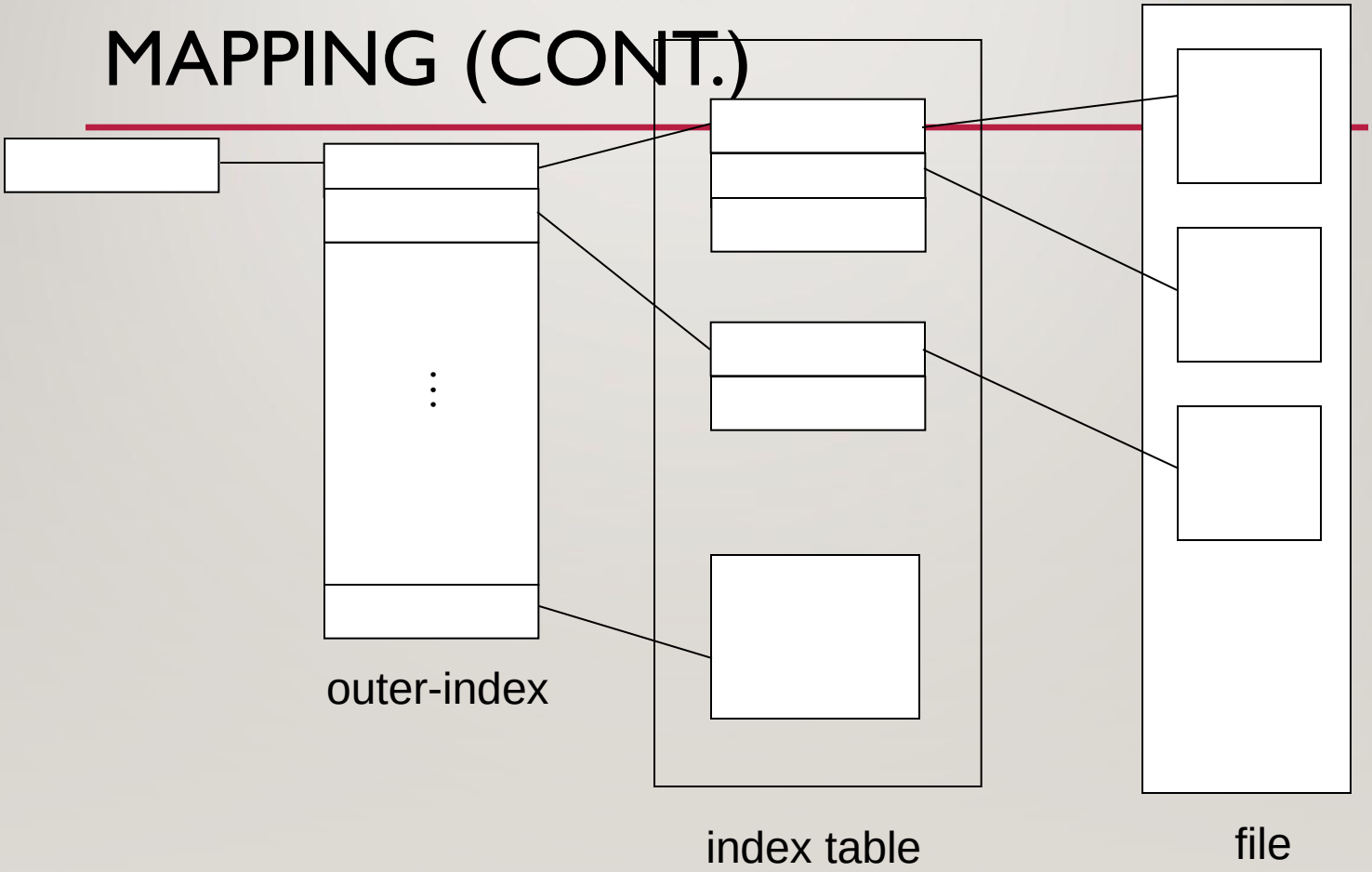


index table

EXAMPLE OF INDEXED

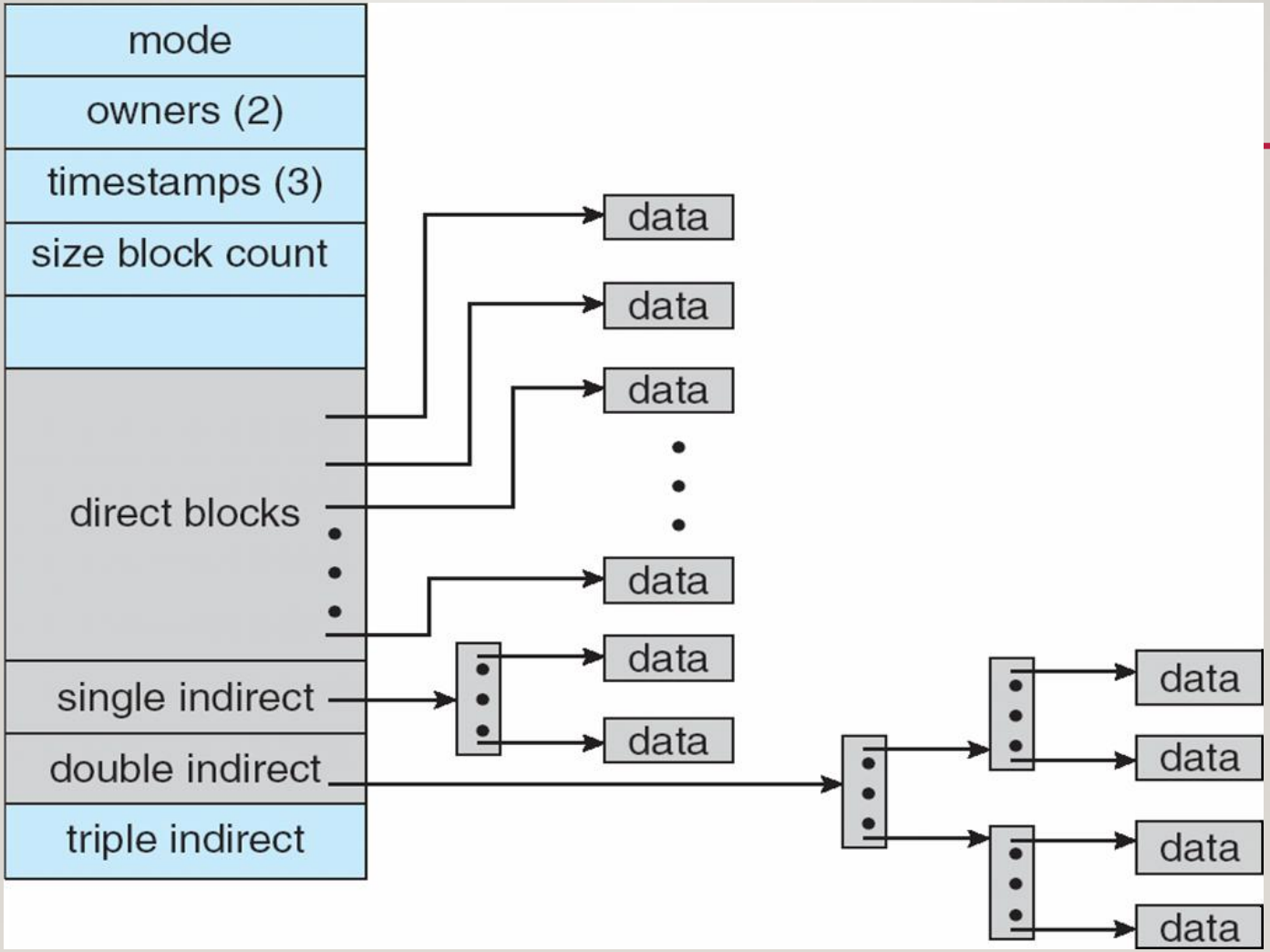


INDEXED ALLOCATION – MAPPING (CONT.)



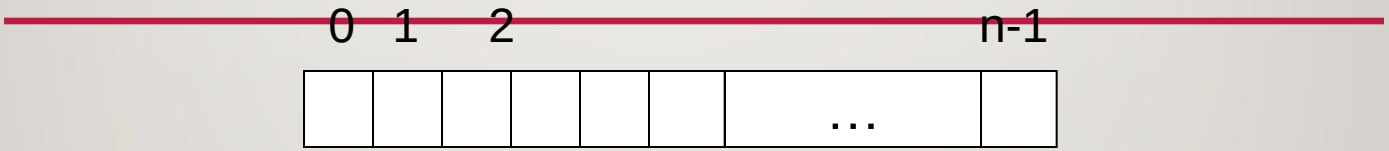
COMBINED SCHEME: UNIX UFS (4K BYTES PER BLOCK)

Dr. Nishant Jain MITS DU Gwalior



FREE-SPACE MANAGEMENT

- Bit vector (*n* blocks)



$$\text{bit}[i] = \begin{cases} 0 \Rightarrow \text{block}[i] \text{ free} \\ 1 \Rightarrow \text{block}[i] \text{ occupied} \end{cases}$$

Block number calculation

(number of bits per word) *
(number of 0-value words) +
offset of first 1 bit

FREE-SPACE MANAGEMENT (CONT.)

- Bit map requires extra space
  - Example:

    block size = 2^{12} bytes

    disk size = 2^{30} bytes (1 gigabyte)

    $n = 2^{30}/2^{12} = 2^{18}$ bits (or 32K bytes)
- Easy to get contiguous files
- Linked list (free list)
  - Cannot get contiguous space easily
  - No waste of space
- Grouping
- Counting

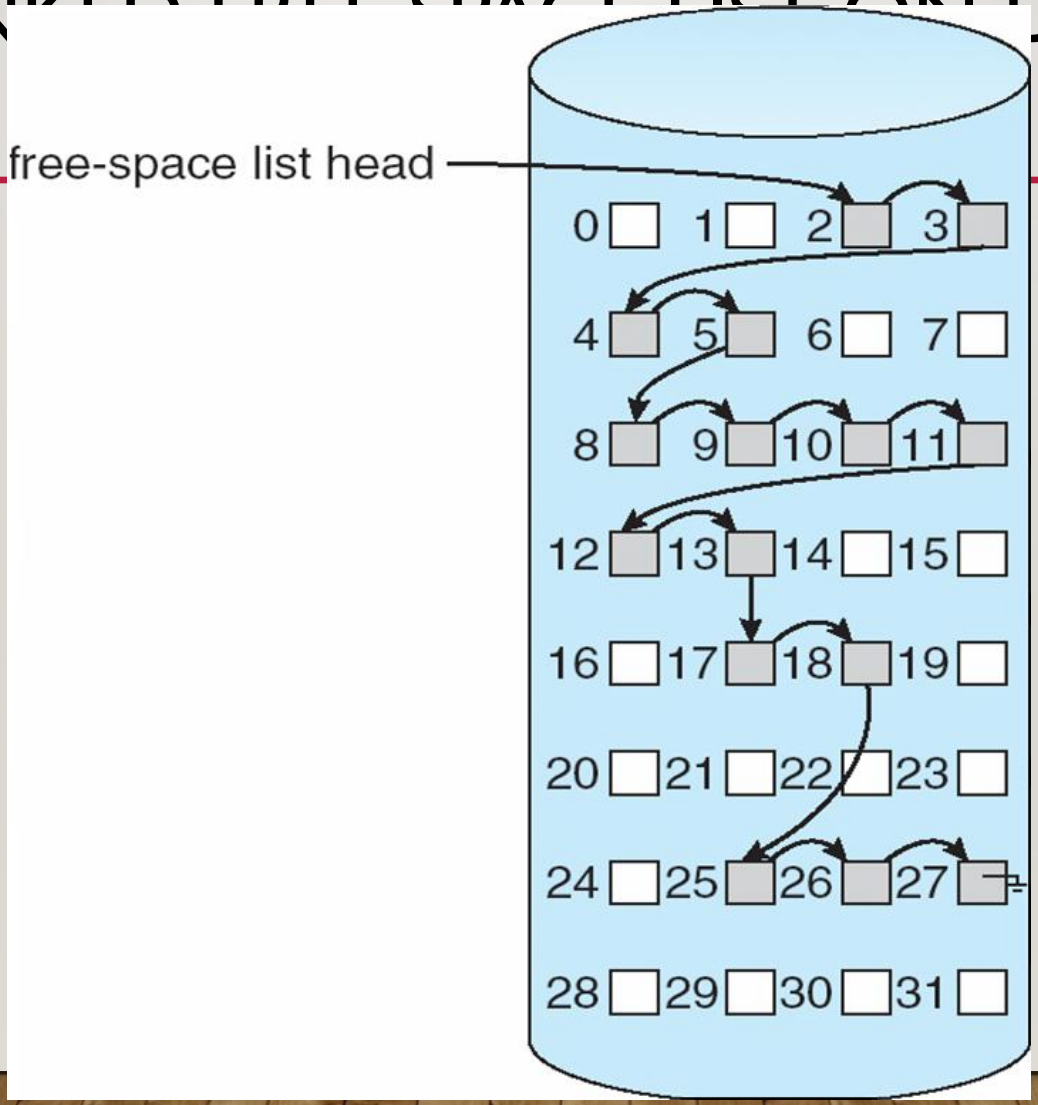
FREE-SPACE MANAGEMENT (CONT.)

- Need to protect:
  - Pointer to free list
  - Bit map
    - Must be kept on disk
    - Copy in memory and disk may differ
    - Cannot allow for block[*i*] to have a situation where bit[*i*] = 1 in memory and bit[*i*] = 0 on disk
  - Solution:
    - Set bit[*i*] = 1 in disk
    - Allocate block[*i*]
    - Set bit[*i*] = 1 in memory

DIRECTORY IMPLEMENTATION

- Linear list of file names with pointer to the data blocks
  - simple to program
  - time-consuming to execute
- Hash Table – linear list with hash data structure
  - decreases directory search time
  - collisions – situations where two file names hash to the same location
  - fixed size

LINKED FREE SPACE LIST ON DISK



EFFICIENCY AND PERFORMANCE

- Efficiency dependent on:
  - disk allocation and directory algorithms
  - types of data kept in file's directory entry
- Performance
  - disk cache – separate section of main memory for frequently used blocks
  - free-behind and read-ahead – techniques to optimize sequential access
  - improve PC performance by dedicating section of memory as virtual disk, or RAM disk

END